SMART MEDICAL WASTE SYSTEM WITH ML BASED PICKUP DELAY PREDICTION

A Project Report

U23CS704 – Mini Project

Submitted by

PAVISHYA P L (61782323102117)

RAM PRIYA M (61782323102129)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

Department of Computer Science and Engineering

SONA COLLEGE OF TECHNOLOGY (AUTONOMOUS), SALEM – 636005

ANNA UNIVERSITY: CHENNAI 600 025

NOVEMBER 2026

SONA COLLEGE OF TECHNOLOGY (AUTONOMOUS)
ANNA UNIVERSITY: CHENNAI – 600 025

BONAFIDE CERTIFICATE

Certified that this project titled **"SMART MEDICAL WASTE SYSTEM WITH ML BASED PICKUP DELAY PREDICTION"** is the bonafide work of **PAVISHYA P L (61782323102117) and RAM PRIYA M (61782323102129)** who carried out the project work **U23CS704 – Mini Project** under my supervision.

SIGNATURE

Ms. A J KEERTHANA, M.E.
SUPERVISOR
Assistant Professor
Department of Computer Science and Engineering
Sona College of Technology,
Salem – 636005

SIGNATURE

Dr. B. SATHIYABHAMA, M.E., Ph.D.
HEAD OF THE DEPARTMENT
Professor and Head
Department of Computer Science and Engineering
Sona College of Technology,
Salem – 636005

Submitted for the Project Viva-Voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

Sona College of Technology

Department of Computer Science and Engineering

U23CS704 – MINI PROJECT – DECLARATION FORM

Declaration by the Student(s)

We hereby declare that the project titled "SMART MEDICAL WASTE SYSTEM WITH ML BASED PICKUP DELAY PREDICTION" is an original work carried out by us under the guidance of Ms. A J Keerthana, Assistant Professor, Department of Computer Science and Engineering, Sona College of Technology, Salem. This project has not been submitted elsewhere for the award of any degree.

We also affirm that all the data and information used in this project are genuine and properly acknowledged. This project work is carried out with reference to the United Nations Sustainable Development Goals Goal 3 – Good Health and Well-being, Goal 6 – Clean Water and Sanitation, Goal 11 – Sustainable Cities and Communities and Goal 12 – Responsible Consumption and Production.

S.No	Register Number	Name	Signature
1.	61782323102117	PAVISHYA P L	
2.	61782323102129	RAM PRIYA M	

Date: _____

Place: Sona College of Technology, Salem

ACKNOWLEDGMENT

First and foremost, we would like to express our sincere gratitude to our **honourable Chairman Sri. C. Valliappa, Vice Chairman Sri. Chocka Valliappa, and Sri. Thyagu Valliappa** for their constant encouragement throughout this course.

We sincerely thank **Dr. S.R.R. Senthil Kumar**, Principal, Sona College of Technology, for his motivation and support.

We wholeheartedly thank **Dr. B. Sathiyabhama**, Professor and Head of the Department of Computer Science and Engineering, for her constant encouragement and support throughout the course.

We take this opportunity to express our deepest gratitude to our project guide, **Ms. A J Keerthana**, Assistant Professor, Department of Computer Science and Engineering, for valuable guidance, continuous support, and encouragement throughout the project.

We would like to thank our parents and friends for their constant support and encouragement. We also express our gratitude to the Almighty for blessings that helped us complete this project successfully.

ABSTRACT

Hospitals produce biomedical waste every day, and this waste must be segregated and handed over to an authorised operator within a limited storage time. In most places, this collection is still arranged over the phone and noted in paper registers, so a delay is noticed only after it has already happened. In this project, we have built the Smart Medical Waste Collection System, a web application used by hospitals, collectors and a central administrator. Hospitals record their colour-coded waste batches (Yellow, Red, White and Blue) and raise pickup requests. A Random Forest classifier looks at eleven operational features of each request, estimates how likely the pickup is to be delayed and marks it as High, Medium or Low delay risk. The administrator then assigns the nearest free collector using Haversine distance, and the collector's live GPS position is shown on the hospital's map through WebSockets until the waste is collected. The system is built with React, FastAPI, SQLite and scikit-learn. On unseen test data, the model gave 70.28 % accuracy and a ROC-AUC of 0.772. Of the requests marked High risk, 82 % were actually delayed, compared with 18 % of the Low-risk ones, and all automated end-to-end tests passed.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	DECLARATION	**iii**
	ACKNOWLEDGMENT	**iv**
	ABSTRACT	**v**
	LIST OF FIGURES	**viii**
	LIST OF TABLES	**ix**
	LIST OF ABBREVIATIONS	**x**
1.	**INTRODUCTION**	**1**
	1.1 Background of the Project	1
	1.2 Problem Statement	1
	1.3 Objectives	2
	1.4 Scope of the Project	2
	1.5 Organisation of the Report	2
2.	**LITERATURE REVIEW**	**3**
	2.1 Review of Related Studies	3
	2.2 Techniques and Technologies Used in Existing Systems	6
	2.3 Research Gap	6
3.	**SYSTEM ANALYSIS**	**7**
	3.1 Existing Techniques	7
	3.2 Limitations of Existing Techniques	7
	3.3 Proposed Technique	7
	3.4 Advantages of the Proposed Technique	8
4.	**SYSTEM DESIGN AND IMPLEMENTATION**	**9**
	4.1 System Architecture	9

Chapter No.	Title	Page No.
	4.2 System Requirements	11
	4.3 System Components / Modules	12
	4.4 Proposed Methodology	16
	4.5 System Workflow	17
	4.6 Algorithms / Techniques Used	17
	4.7 Dataset / Data Collection	18
	4.8 Data Preprocessing	20
	4.9 Implementation	20
5.	**RESULTS AND DISCUSSION**	**21**
	5.1 Experimental Setup	21
	5.2 Evaluation Metrics	21
	5.3 Experimental Results	22
	5.4 Comparison with Existing Techniques	25
6.	**CONCLUSION AND FUTURE SCOPE**	**26**
	REFERENCES	**27**
	APPENDIX 1 – SUSTAINABLE DEVELOPMENT GOALS (SDG)	**29**
	APPENDIX 2 – SOURCE CODE	**30**
	APPENDIX 3 – APPLICATION SCREENSHOTS	**31**

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	Overall System Architecture	10
4.2	Request Authorisation Pipeline	13
4.3	Pickup Request Lifecycle	15
4.4	Live GPS Streaming Sequence	16
4.5	System Workflow from Waste Recording to Collection	17
4.6	Dataset Class Distribution and Delay Rate by Collector Availability	19
5.1	Confusion Matrix on the Test Set	22
5.2	Feature Importance of the Random Forest Model	23
A.1	Landing Page	31
A.2	Hospital – Schedule Pickup with ML Delay-Risk Preview	31
A.3	Hospital – Live Collector Dispatch Tracking	32
A.4	Admin – Assign Nearby Collector	32
A.5	Collector – Active Medical Waste Dispatch	32

LIST OF TABLES

Table No.	Title	Page No.
2.1	Summary of Related Studies	5
3.1	Comparative Analysis of Collection Approaches	8
4.1	Hardware Requirements	11
4.2	Software Requirements	12
4.3	User Roles and Responsibilities	13
4.4	Biomedical Waste Colour Categories	14
4.5	Feature Description of the Dataset	19
4.6	Database Tables	20
5.1	Experimental Setup	21
5.2	Test-Set Performance of the Random Forest Model	22
5.3	Actual Delay Rate per Predicted Risk Tier	23
5.4	Automated End-to-End Test Results	24
5.5	Performance and Latency Metrics (median of 15 runs)	24
5.6	Comparison of Classifiers on the Test Set	25

LIST OF ABBREVIATIONS

API	Application Programming Interface
AUC	Area Under the Curve
BMW	Bio-Medical Waste
CPCB	Central Pollution Control Board
CSV	Comma-Separated Values
FN	False Negative
FP	False Positive
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
JSON	JavaScript Object Notation
JWT	JSON Web Token
KNN	K-Nearest Neighbours
ML	Machine Learning
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SDG	Sustainable Development Goal
TN	True Negative
TP	True Positive
WHO	World Health Organization

CHAPTER 1

INTRODUCTION

1.1 Background of the Project

Hospitals, clinics, laboratories and blood banks produce biomedical waste every day. This includes soiled dressings and body parts, contaminated plastics such as tubes and gloves, sharp items such as needles and blades, and broken glass. About 15 % of health-care waste is hazardous. If it is not handled properly, it can spread infection, injure workers and pollute water and soil [23]. Studies on medical waste show that the risk does not depend only on how the waste is treated; it also depends on how well it is segregated, stored and transported [24].

1.2 Problem Statement

Hospitals and collection operators who still coordinate pickups manually face the following problems:

- No digital record of waste batches – the category, type and quantity are written in registers, so it takes a lot of time to find the totals for each category or each hospital when reports have to be submitted.
- No early warning of delays – urgent requests (waste stored for a long time, high-risk categories, few free collectors, peak traffic hours) are handled in the same first-come order as normal ones, and the 48-hour storage limit can be crossed without anyone noticing.
- Collector is chosen without distance details – the dispatcher picks a collector from memory and not the nearest free vehicle, so the travel time increases.
- No tracking during transit – once a vehicle is sent, the hospital does not know where it is or when it will reach.

1.3 Objectives

- To give hospitals, collectors and the administrator their own secure portals, with role-based access control checked on the server.
- To train and use a machine learning model that predicts the chance of a pickup getting delayed and marks each request as High, Medium or Low delay risk.
- To list collectors by their Haversine distance from the hospital and their availability, so that the administrator can assign the nearest free collector.

1.4 Scope of the Project

This system is meant for a regional biomedical waste collection network made up of several hospitals, a group of collection vehicles and a supervising authority. It runs as a web application with a FastAPI backend and an SQLite database, which can later be moved to PostgreSQL. The scope includes registration, waste recording, pickup requests with delay-risk prediction, nearby collector assignment, live GPS tracking, collection history and analytics. Route optimisation, IoT bin sensors and mobile apps are kept for the Future Scope.

1.5 Organisation of the Report

Chapter 2 reviews the related literature, Chapter 3 studies the existing process and our proposed approach, Chapter 4 explains the design and implementation, Chapter 5 discusses the results and Chapter 6 gives the conclusion and future scope. The references are given after that, followed by the appendices on the Sustainable Development Goals, the source code and the application screenshots.

CHAPTER 2

LITERATURE REVIEW

2.1 Review of Related Studies

This section reviews the studies, rules and standards on which this project is based. We have grouped them into five themes, which together explain the background and the requirements of the proposed system.

Regulation and practice of biomedical waste management. The WHO handbook [23] and the Bio-Medical Waste Management Rules, 2016 [17] both treat collection as a controlled and documented step between segregation at the source and treatment. The rules fix the colour categories and the 48-hour limit, and the CPCB guidelines [4] turn these into clear requirements for containers, storage areas, labelling and authorised transport. However, the field study by Datta, Mohi and Chander [5] and the review by Windfeld and Brooks [24] show that, in practice, these rules are not followed well because of poor record keeping, irregular collection and the lack of monitoring tools. From these studies, it is clear that a practical system must follow the colour categories exactly, record every batch with its time and source, and show delays before the storage limit is crossed.

Smart and IoT-based waste collection. Hannan et al. [11] and Anagnostopoulos et al. [2] explain how RFID, GPS, GSM and IoT sensors have been connected to central servers to monitor bins and vehicles in city waste collection. These works prove that sharing data in real time between vehicles and a central server is possible and useful. At the same time, both surveys point out that most of these systems only display data. They show where the vehicles are, but they do not tell the operator which request is the most urgent or which vehicle should be sent.

Machine learning for waste and logistics prediction. Abdallah et al. [1] found that artificial intelligence in waste management has mostly been used to forecast how much waste will be produced, and Kannangara et al. [15] showed that decision trees and neural networks can give good regional forecasts from simple operational and socio-economic data. Govindan et al. [9] showed during the COVID-19 period that medical waste volumes and risks can change very quickly, and that collection decisions must keep up with these changes. However, none of these works predicts the delay of a single pickup request. The Random Forest [3], the statistical methods explained by Hastie, Tibshirani and Friedman [12], cost-sensitive learning for imbalanced classes [13] and stratified cross-validation [16] give us the techniques to build and test such a predictor. scikit-learn [18] makes it easy to use the same pipeline inside a web service, and ROC analysis [6] tells us how well the predicted probabilities rank the risky requests.

Distance, routing and vehicle tracking. The haversine formula [21] gives an accurate great-circle distance between two GPS points and is commonly used to find the nearest location. Vehicle routing research [22] gives methods to plan efficient routes with many stops, but these methods assume that the vehicle positions and the urgency of each request are already known. Trajectory data mining [25] shows that stored GPS traces can later be used to estimate travel time and traffic. OpenStreetMap [10] gives free map data for showing vehicles, and the WebSocket protocol [7] can send positions to many users at the same time with very little overhead.

Secure web application architecture. REST [8] is the most widely used style for building web services because it is stateless and has a simple, uniform interface. Role-based access control [20] gives permissions to roles instead of to individual users, which fits well for a system that has three separate user groups. JSON Web Tokens [14] carry the user’s identity and role between the

client and the server, and bcrypt [19] protects the stored passwords from brute-force attacks.

Overall, the literature gives us tested building blocks – the regulatory categories and time limits, real-time tracking, machine learning classifiers, nearest-location search and secure web standards – but each study uses them separately. The systems we reviewed either record and track collection without predicting delays, or predict waste volumes without helping in daily assignment and tracking. This gap led us to build one combined system that predicts the delay risk of every request, helps the administrator assign the nearest free collector and shows the vehicle to the hospital in real time.

Table 2.1 gives a summary of the studies that are most closely related to our system, listed in the order of their reference numbers.

Table 2.1 Summary of Related Studies

Ref.	Authors / Year	Focus	Relevance / Limitation
[1]	Abdallah et al., 2020	AI in solid waste management	Models rarely deployed in operational systems
[2]	Anagnostopoulos et al., 2017	IoT-enabled smart waste collection	Dynamic scheduling remains open
[3]	Breiman, 2001	Random Forest classifier	Basis of the delay-risk model
[5]	Datta et al., 2018	BMW management in India	Gaps in monitoring and timely collection
[9]	Govindan et al., 2021	Medical waste collection model	Optimisation model; no live system
[11]	Hannan et al., 2015	Waste monitoring technologies	Tracking without decision support
[15]	Kannangara et al., 2018	ML for waste generation	Forecasts volumes, not pickup delays
[17]	MoEFCC, 2016	BMW Management Rules	Colour categories, 48-hour limit
[24]	Windfeld & Brooks, 2015	Medical waste management review	Transport and storage are key risks

2.2 Techniques and Technologies Used in Existing Systems

- Paper manifests and phone calls – the most common method; there is no central record, no priority for urgent requests and no way to see the vehicles.
- Spreadsheet logs – totals can be calculated, but the data is entered later and is not shared with collectors in real time.
- Fleet GPS trackers – show the vehicle positions to the operator [11], but they are not linked to hospital requests and the hospital cannot see them.
- Optimisation and forecasting models [9], [15] – estimate waste volumes or plan routes in advance, but they are rarely used in daily request handling.
- IoT smart-bin systems [2] – detect how full the city bins are; they need special hardware and do not handle the regulated handover from hospital to operator.

2.3 Research Gap

From the review, the following gaps were identified:

- The existing tools record or track collection, but they do not predict which pickup requests may get delayed at the time the requests are raised.
- Collector assignment rarely uses distance and availability details that are shown to the dispatcher.
- The live position of the vehicle is not shared with the hospital that is waiting for the pickup.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing Techniques

In the existing process, the hospital staff segregate the waste into coloured bags and write the quantities in a register. When the storage area is full, the hospital calls the collection operator. The operator's dispatcher notes down the request and chooses a vehicle based on what they remember about where the vehicles are. The driver collects the waste and signs a paper manifest [4]. The hospital gets no information about the vehicle until it arrives, and the authority gets the totals only through periodic reports that are prepared by hand [5].

3.2 Limitations of Existing Techniques

- Requests are handled in the order in which they come, even if some of them are more urgent or risky.
- There is no warning when a request is likely to be delayed beyond the allowed storage period.
- The nearest free collector is not known, so the travel distance and time are more than needed.
- The hospital cannot see where the vehicle is or when it will arrive.

3.3 Proposed Technique

The proposed system makes the whole collection cycle digital and uses the following techniques:

- Colour-coded waste recording – every batch is saved with its category (Yellow, Red, White or Blue), waste type, quantity in kilograms and notes, and is linked to the hospital that produced it.

- Machine learning delay-risk prediction – when a pickup is requested, a Random Forest pipeline estimates the chance of delay from eleven features (category, quantity, storage hours, waiting hours, risk level, distance, previous delay, request frequency, collector availability, day and hour) and maps it to High (≥ 0.65), Medium (0.35–0.65) or Low (< 0.35) risk.
- Assignment by the admin based on distance – hospitals cannot choose collectors; the administrator sees all collectors sorted by availability and Haversine distance and assigns the nearest free one.

3.4 Advantages of the Proposed Technique

Table 3.1 compares the proposed technique with the existing methods on the main features of collection management.

Table 3.1 Comparative Analysis of Collection Approaches

Feature	Paper / Phone	Spreadsheet Log	Fleet GPS Tracker	Proposed System
Waste batch record	Register	Manual entry	No	Per batch, colour-coded
Urgency of request	Not assessed	Not assessed	Not assessed	ML delay-risk tier
Collector selection	From memory	From memory	Map view only	Ranked by distance
Hospital sees vehicle	No	No	No	Live map (WebSocket)
Step time stamps	Partial	Manual	Vehicle only	Request to completion
Access control	None	File sharing	Operator only	JWT + three roles
Analytics	Manual	Manual formulas	Distance reports	Live dashboards

- High-risk requests are marked and shown first, so the administrator can act before the storage limit is reached.
- The nearest free collector is assigned, which reduces the travel distance.
- Hospitals know where the vehicle is and when it will reach them.

CHAPTER 4

SYSTEM DESIGN AND IMPLEMENTATION

4.1 System Architecture

The system uses a layered client–server architecture (Figure 4.1). The presentation layer is a React single-page application built with Vite and styled using Tailwind CSS. It talks to the backend through REST [8] calls that carry a JWT bearer token, and through a WebSocket [7] channel for live locations. The application layer is a FastAPI service running on Uvicorn. It checks the token and the user's role, validates the request data using Pydantic and carries out the main logic. It uses SQLAlchemy to work with the SQLite database and loads the trained scikit-learn pipeline once at start-up to make predictions.

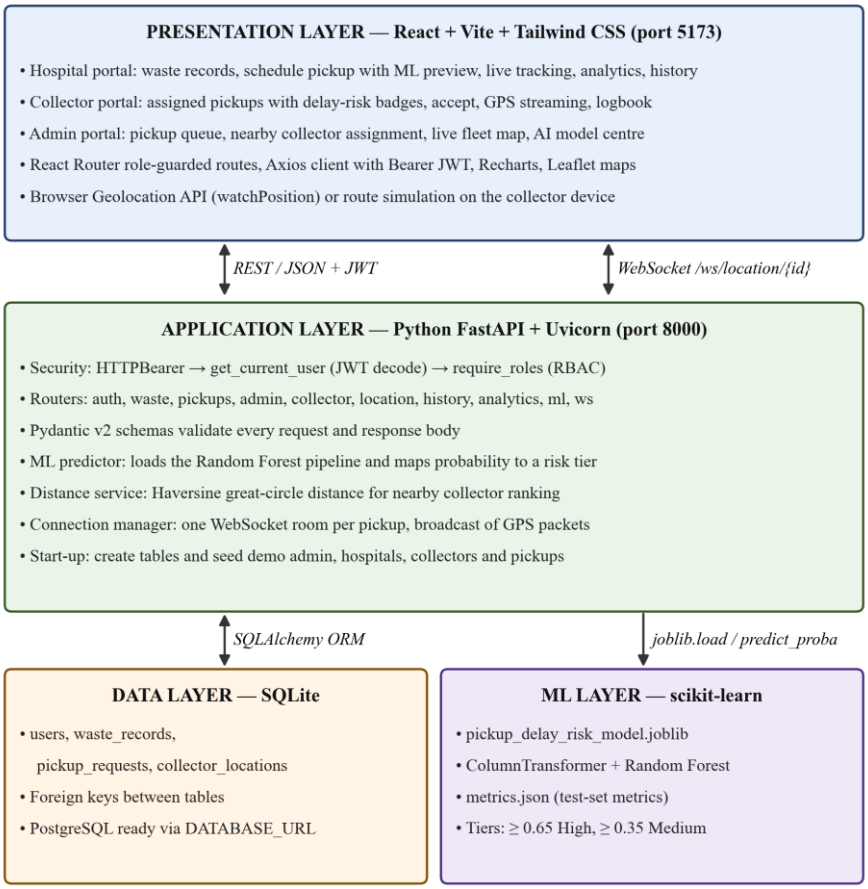


Figure 4.1 Overall System Architecture

Figure 4.1 shows the three role-based portals in the presentation layer, the FastAPI application layer with its routers and services, the SQLite data layer and the scikit-learn model used to predict delay risk.

As shown in Figure 4.1, the system has four layers. The presentation layer has three role-based portals. The hospital portal is used for waste records, pickup scheduling with the ML preview, live tracking and history. The collector portal is used for assigned pickups, accepting them and sending GPS positions. The admin portal is used for the pickup queue, nearby collector assignment, the live fleet map and the AI model centre. The collector's device gives its position through the browser Geolocation API or through a route simulation.

The portals talk to the application layer using REST/JSON requests that carry the JWT, and using a WebSocket connection on /ws/location/{id} for live positions. In the FastAPI application layer, every request first goes

through the security checks (get_current_user and require_roles) and only then reaches the routers. Here, the ML predictor turns a delay probability into a risk tier, the distance service sorts collectors using the Haversine formula and the connection manager sends out the GPS packets. Through SQLAlchemy, it uses the SQLite data layer with four tables, and it loads the saved scikit-learn pipeline from the ML layer to call predict_proba for every new request, using the 0.65 and 0.35 thresholds.

4.2 System Requirements

This section lists the hardware and software needed to develop, run and use the system. The hardware requirements are given in Section 4.2.1 and the software requirements are given in Section 4.2.2.

4.2.1 Hardware Requirements

Table 4.1 lists the minimum hardware needed to run the system.

Table 4.1 Hardware Requirements

Component	Specification
Processor	Intel Core i3 / AMD Ryzen 3 or higher (dual core, 2.0 GHz)
RAM	4 GB minimum (8 GB recommended for model training)
Storage	2 GB available disk space
Network	Local network or localhost; Internet for map tiles
Client device	Desktop browser, or smartphone with GPS for the collector

4.2.2 Software Requirements

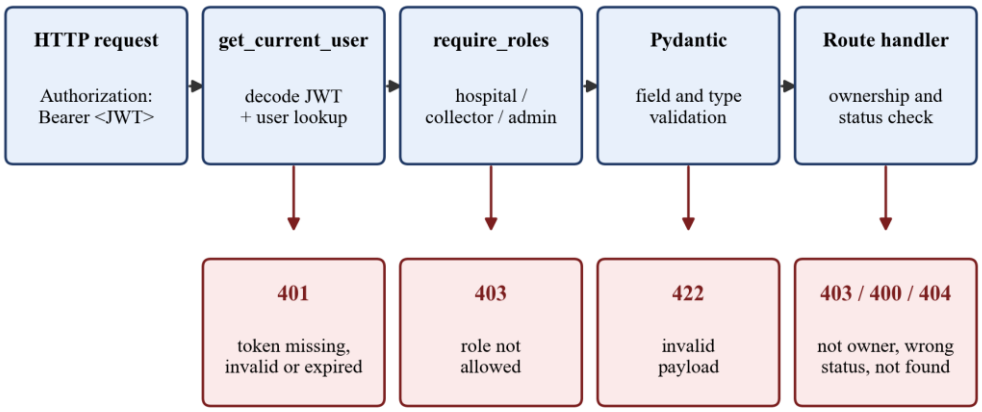
Table 4.2 lists the software and libraries used to develop and run the system.

Table 4.2 Software Requirements

Category	Software / Version
Operating System	Windows 10 / 11, Linux (Ubuntu 20.04+) or macOS
Runtimes	Python 3.10 or later (tested on 3.14); Node.js 18 or later with npm
Frontend	React 19, Vite 8, Tailwind CSS 4, React Router 7, Axios, Recharts, Leaflet / React-Leaflet, Lucide icons, React Hot Toast
Backend	FastAPI, Uvicorn, SQLAlchemy 2, Pydantic 2, python-jose (JWT), bcrypt, websockets
Database	SQLite (PostgreSQL compatible through DATABASE_URL)
Machine Learning	scikit-learn, pandas, NumPy, joblib
Testing	pytest with FastAPI TestClient (httpx)
IDE / Browser	Visual Studio Code; Chrome, Edge or Firefox

4.3 System Components / Modules

4.3.1 Module 1: Authentication and Role-Based Access Control. Users register as a hospital or a collector by giving their name, e-mail, phone number, address and map location; the administrator account is created when the database is first set up. Passwords are hashed using bcrypt [19] with a cost factor of 12 and are never stored as plain text. When a user logs in, the server gives an HMAC-SHA256 signed JWT [14] that holds the user ID and is valid for 24 hours. Every protected endpoint uses get_current_user, which reads the token and loads the user, and require_roles, which blocks users whose role is not allowed [20] (Figure 4.2). The data is also filtered by owner, so a hospital sees only its own waste and pickups, and a collector sees only the pickups given to it. Table 4.3 lists the roles.



Hospitals and collectors see only records whose hospital_id or collector_id equals their own user id.

Figure 4.2 Request Authorisation Pipeline

Figure 4.2 shows how every request goes through token checking, role checking, data validation and owner and status checks, and how a failure at any step stops the request with a 401, 403, 422, 400 or 404 response.

Table 4.3 User Roles and Responsibilities

Role	Responsibilities
Hospital	Records waste batches; raises pickup requests and previews the predicted delay risk; tracks the assigned collector live; views analytics and collection history; exports history as CSV.
Collector	Views pickups assigned by the administrator with delay-risk badges; accepts a pickup; streams GPS location (device GPS or simulation); confirms handover; views personal logbook.
Admin	Monitors system-wide statistics; reviews the pending queue sorted by delay risk; assigns the nearest available collector; monitors the live fleet; inspects ML model metrics; manages hospital and collector directories and history.

4.3.2 Module 2: Biomedical Waste Recording. The hospital records each batch with its colour category, a description of the waste type, the quantity and unit (kilograms) and optional storage notes. The category decides the hazard risk level that is used by the model: Yellow and White are level 4, Red is level 3 and Blue is level 2, based on how hazardous each type of waste is as per the rules [17]. Table 4.4 lists the categories. Every record has an is_scheduled flag, so the same batch cannot be scheduled for pickup twice.

Table 4.4 Biomedical Waste Colour Categories

Category	Typical Waste	Treatment	Risk Level
Yellow	Anatomical waste, soiled dressings, expired medicines, cultures	Incineration / deep burial	4
Red	Contaminated plastics: tubing, catheters, gloves, urine bags	Autoclaving / shredding, recycling	3
White	Sharps: needles, scalpels, blades	Autoclaving / dry heat, encapsulation	4
Blue	Broken glass vials, ampoules, metallic implants	Disinfection, glass recycling	2

4.3.3 Module 3: Pickup Request and Delay-Risk Prediction. When the hospital schedules a pickup for a recorded batch, the server first checks that the batch belongs to that hospital and has not already been scheduled. It then works out the features for the model: how long the batch has been stored, the risk level of its category, how many requests the hospital made in the last seven days, the number of collectors, and the current day of the week and hour. The predictor gives the probability of delay, which is converted into a risk tier using Equation 4.1. The request is saved with the status Pending, along with the risk tier, the probability, the model version and the prediction time. Before submitting, the hospital can see the same prediction as a live preview.

$$Risk = High\ if\ p \geq 0.65;\quad Medium\ if\ 0.35 \leq p < 0.65;\quad Low\ if\ p < 0.35 \tag{4.1}$$

Pending requests are shown to the administrator in order of risk (High, then Medium, then Low) and, within the same tier, by request time, so the requests most likely to be delayed are handled first.

4.3.4 Module 4: Nearby Collector Assignment. For a selected pending request, the administrator opens the assignment window. The server takes the latest GPS position of each collector (or the registered location if no position is saved), finds the Haversine distance [21] to the hospital and marks the collector as busy if it already has an accepted pickup. Free collectors are listed first, and then all collectors are sorted by distance. Only the administrator can

assign a collector, and the collector can be changed while the request is still Pending or Assigned. Figure 4.3 shows the pickup lifecycle followed by the server.

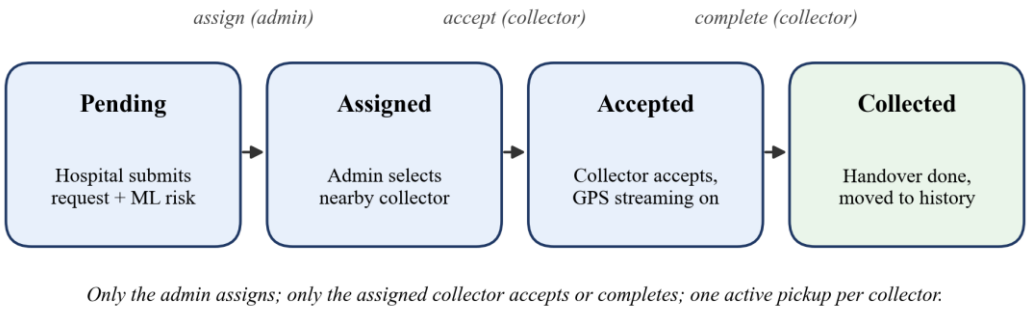


Figure 4.3 Pickup Request Lifecycle

Figure 4.3 shows how a request moves from Pending to Assigned, Accepted and Collected through endpoints that are allowed only for the right role; wrong status changes are rejected with HTTP 400.

4.3.5 Module 5: Live GPS Tracking. When the collector accepts a pickup, its status becomes Accepted and the active pickup screen opens. The browser's Geolocation API (watchPosition) gives the position of the device; for demonstrations on a desktop, a simulation mode moves the marker step by step towards the hospital. Each position is sent to the server, which saves it in the collector_locations table and sends a LOCATION_UPDATE packet to every WebSocket client connected to /ws/location/{pickup_id} (Figure 4.4). A client that connects later gets the last known position at once. The hospital map, which uses OpenStreetMap tiles [10], updates without reloading the page through the WebSocket protocol [7], and the administrator's fleet map refreshes all active pickups every five seconds. Location updates are accepted only while the pickup is in the Accepted state, so tracking stops as soon as the pickup is completed.

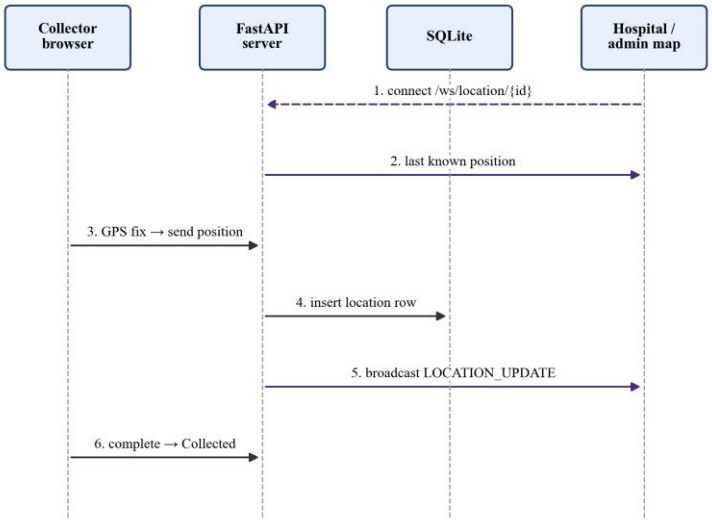


Figure 4.4 Live GPS Streaming Sequence

Figure 4.4 shows how the hospital map joins the WebSocket room of the pickup and how each GPS update from the collector is saved and sent to all listeners until the pickup is completed.

4.3.6 Module 6: Collection History and Analytics. When the collector confirms the handover, the status becomes Collected and the completion time is saved. Collected pickups are removed from all active lists and are shown only in the collection history, which is filtered by role and can be exported as CSV for compliance records [4]. The analytics endpoints calculate the total waste recorded and collected, the number of pending, assigned, active and completed pickups, the waste in each colour category, the requests in each delay-risk tier and the daily waste trend. Hospitals see only their own figures, while the administrator sees the totals for the whole system. The administrator also has an AI Model and Metrics Centre that shows the model's settings, test results, confusion matrix and feature importances taken from metrics.json.

4.4 Proposed Methodology

We followed an iterative method for this project. The requirements were taken from the Bio-Medical Waste Management Rules and the problem statement. We first designed the database using SQLAlchemy models. Next, we built the machine learning part separately: we generated a dataset, trained

and tested a model, and saved the trained pipeline along with its results. The REST API was then built around the pickup lifecycle and connected with the predictor and the distance service. After that, we added the WebSocket channel and the map screens, and finally the analytics and history. Each stage was checked using automated end-to-end tests that run the complete workflow through the API.

4.5 System Workflow

Figure 4.5 shows the complete workflow, from the time a waste batch is recorded to the time it appears in the collection history.

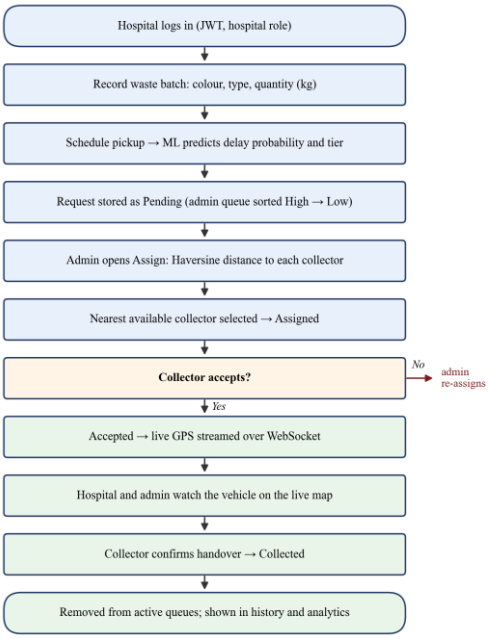


Figure 4.5 System Workflow from Waste Recording to Collection

Figure 4.5 shows the flowchart that follows a batch through recording, the pickup request with ML prediction, assignment by the admin, acceptance by the collector, live tracking and completion.

4.6 Algorithms / Techniques Used

4.6.1 Random Forest Classifier

A Random Forest [3] builds B decision trees. Each tree is trained on a random sample of the training data and uses a random set of features at every split. For a request x, each tree t gives the fraction of delayed samples in the

leaf that x reaches, and the forest takes the average of these values (Equation 4.2). Our model uses B = 150 trees with a maximum depth of 12, at least 4 samples to split a node and 2 samples in each leaf. Balanced class weights [13] give the smaller class (delayed pickups) the same total weight as the larger class.

$$p(delay \mid x) = (1 / B) \cdot \Sigma_{t=1..B} \; p_t(delay \mid x) \tag{4.2}$$

4.6.2 Haversine Distance

The great-circle distance d between the hospital ($\varphi 1, \lambda 1$) and a collector ($\varphi 2, \lambda 2$), with latitude and longitude in radians and the Earth's radius R = 6371 km, is found using Equation 4.3.

$$a = \sin^2(\Delta\varphi/2) + \cos\varphi 1 \cdot \cos\varphi 2 \cdot \sin^2(\Delta\lambda/2), \quad d = 2R \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{(1 - a)}) \tag{4.3}$$

4.6.3 Nearby Collector Ranking Algorithm

For a pending request, we take the latest position of every collector (or its registered location), find its Haversine distance to the hospital, and mark the collector as busy if it already has an accepted pickup. The list is then sorted so that free collectors come first and, within each group, the nearest one comes first. So, the first entry in the list is always the nearest free collector.

4.7 Dataset / Data Collection

Real records of biomedical waste pickups are confidential and are not available to the public. So, we generated a synthetic dataset of 1,800 pickup records (generate_dataset.py, random seed 42) using distributions that match how collection works in practice. Waste categories were chosen with probabilities 0.35 (Yellow), 0.35 (Red), 0.15 (White) and 0.15 (Blue); the quantity follows a gamma distribution between 2 and 180 kg; storage and waiting times follow exponential distributions; distance follows a normal distribution between 0.5 and 35 km; collector availability ranges from 1 to 5;

and the hour of the day is weighted towards working hours. The delay label is taken from a logistic function of a hidden score that goes up with waiting time, storage time, distance, quantity, previous delays, peak hours, weekends and hazard level, and goes down when more collectors are available. The final dataset has 785 delayed (43.6 %) and 1015 on-time (56.4 %) records (Figure 4.6). Table 4.5 describes the features.

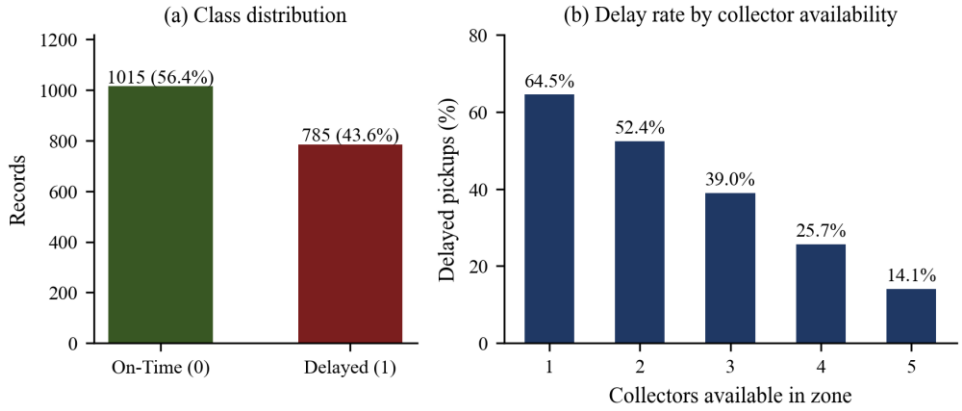


Figure 4.6 Dataset Class Distribution and Delay Rate by Collector Availability

Figure 4.6 shows in part (a) that the dataset is moderately imbalanced, and shows in part (b) that the share of delayed pickups drops sharply when more collectors are available in the zone.

Table 4.5 Feature Description of the Dataset

Feature	Type	Range / Values
waste_category	Categorical	Yellow, Red, White, Blue
waste_quantity	Numerical	2 – 180 kg
storage_duration_hours	Numerical	1 – 72 h
request_waiting_hours	Numerical	0.2 – 48 h
waste_risk_level	Numerical	2 – 4
pickup_distance_km	Numerical	0.5 – 35 km
previous_delay_hours	Numerical	0 – 24 h
hospital_request_frequency	Numerical	1 – 9 requests per week
collector_availability	Numerical	1 – 5 collectors
day_of_week	Numerical	0 (Monday) – 6 (Sunday)
time_of_day	Numerical	0 – 23 h
is_delayed (target)	Binary	0 = On-Time, 1 = Delayed

4.8 Data Preprocessing

- Train–test split – the data is split into 80 % for training (1,440 records) and 20 % for testing (360 records), keeping the same share of delayed pickups in both sets [16].
- Missing values – missing numbers are filled with the median and the missing category with the most common value, so the model can still work when some input is missing.
- Single pipeline – filling missing values, scaling, encoding and the classifier are combined in one scikit-learn Pipeline [18] and saved using joblib, so the same steps are used both in training and in the API.

4.9 Implementation

The database has four tables, which are summarised in Table 4.6. Users of all three roles are stored in the users table, and the role column tells them apart. A pickup request is linked to the hospital and, once assigned, to the collector, and it is linked one-to-one with the waste batch it collects.

Table 4.6 Database Tables

Table	Key Attributes
users	id (PK), name, email (UQ), phone, password_hash, role (hospital / collector / admin), address, latitude, longitude, created_at
waste_records	id (PK), hospital_id (FK), category, waste_type, quantity, unit, notes, is_scheduled, created_at
pickup_requests	id (PK), hospital_id (FK), collector_id (FK, nullable), waste_record_id (FK), status, delay_risk, delay_probability, prediction_timestamp, model_version, pickup_address, hospital_latitude, hospital_longitude, storage_duration_hours, requested_at, assigned_at, accepted_at, completed_at
collector_locations	id (PK), collector_id (FK), pickup_id (FK), latitude, longitude, updated_at

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Experimental Setup

All experiments were run on the development computer described in Table 5.1. The machine learning experiments used the 20 % stratified test set, which the model never saw during training.

Table 5.1 Experimental Setup

Parameter	Value
Operating system	Windows 11 (64-bit)
Python / Node.js	Python 3.14.3; Node.js 24
Key libraries	FastAPI, SQLAlchemy 2, scikit-learn, pandas, NumPy, pytest
Dataset	1,800 records; 1,440 train / 360 test (stratified)
Model	RandomForestClassifier (150 trees, depth 12, balanced weights)
Baselines	Logistic Regression, Gaussian Naive Bayes, KNN (k = 7), Decision Tree (depth 8)
Validation	Hold-out test set and stratified 5-fold cross-validation
Database	SQLite, seeded with 1 admin, 3 hospitals, 3 collectors and sample pickups

5.2 Evaluation Metrics

Here, TP, TN, FP and FN stand for true positives, true negatives, false positives and false negatives for the Delayed class. The following metrics were used:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad (5.1)$$

$$Precision = TP / (TP + FP), \qquad Recall = TP / (TP + FN) \quad (5.2)$$

$$F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall) \quad (5.3)$$

We also used ROC-AUC [6], which shows how well the predicted probabilities place delayed requests above on-time ones, without depending

on any one threshold. Since the risk tiers are taken from the probability, a good ranking matters more for this application than accuracy at a single threshold.

5.3 Experimental Results

5.3.1 Delay-Risk Model Performance

Table 5.2 shows how the final Random Forest model performed on the unseen test set, and Figure 5.1 shows its confusion matrix. The model correctly found 101 of the 157 delayed pickups and 152 of the 203 on-time pickups.

Table 5.2 Test-Set Performance of the Random Forest Model

Class	Precision	Recall	F1-Score	Support
On-Time	73.08 %	74.88 %	73.97 %	203
Delayed	66.45 %	64.33 %	65.37 %	157
Weighted average	70.19 %	70.28 %	70.22 %	360
Accuracy			70.28 %	360
ROC-AUC			0.7717	360

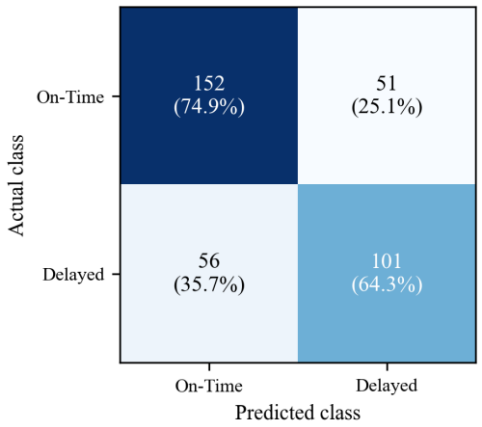


Figure 5.1 Confusion Matrix on the Test Set

Figure 5.1 shows the actual class in rows and the predicted class in columns; the percentages are calculated for each actual class.

Figure 5.2 shows the Gini feature importances of the model. Storage duration, request waiting time and collector availability are the three most important features, followed by previous delay and distance. This matches the real causes of delay described in Section 4.7. The colour category has less

direct effect because most of its effect is already captured by the risk-level feature.

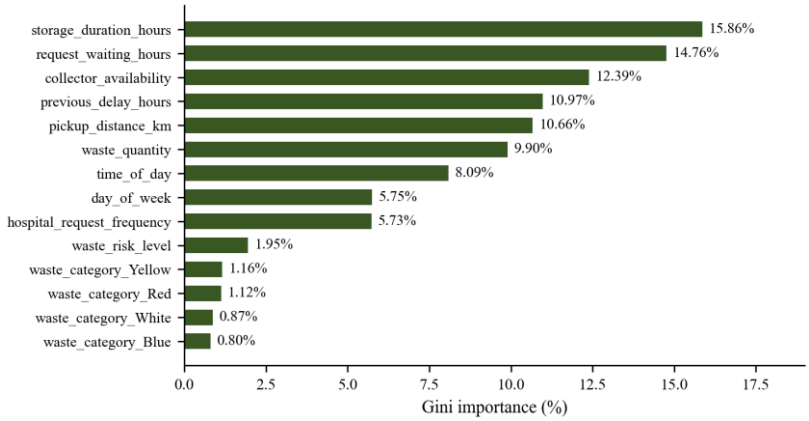


Figure 5.2 Feature Importance of the Random Forest Model

Figure 5.2 shows that the time-related and capacity-related features have the most effect on the prediction, while each one-hot category column has only a small effect.

5.3.2 Reliability of the Risk Tiers

To check whether the three tiers are meaningful, we grouped the test requests by the tier given by the final model and found how many in each group were actually delayed (Table 5.3). The share of delayed pickups goes up steadily from the Low tier to the High tier, so the tiers can be used to order the administrator's queue.

Table 5.3 Actual Delay Rate per Predicted Risk Tier

Risk Tier	Probability Range	Requests	Actually Delayed
Low	$p < 0.35$	113	17.7 %
Medium	$0.35 \leq p < 0.65$	179	45.3 %
High	$p \geq 0.65$	68	82.4 %

5.3.3 Automated Functional Testing

The backend test suite (backend/tests/test_api.py) resets the database, adds the demo data and runs the complete workflow through the HTTP API. All five test functions, with 54 assertions in total, passed (Table 5.4).

Table 5.4 Automated End-to-End Test Results

S.No	Test	What is Verified	Assertions	Result
1	test_root_endpoint	Service health response	2	Passed
2	test_admin_login	bcrypt verification, JWT issue, admin role	3	Passed
3	test_hospital_register_and_work flow	Register, record waste, pickup with ML risk, nearby search, assign, accept, GPS update, complete, removal from active list, history	24	Passed
4	test_ml_prediction_and_metrics _endpoints	Prediction fields and ranges, legacy endpoint, model information, test metrics	18	Passed
5	test_analytics_endpoints	Overview, category and delay-risk distributions	7	Passed
	Total		54	5 / 5 passed

5.3.4 Performance and Latency

Table 5.5 shows the median server-side time taken by the main operations. Login is the slowest because bcrypt with a cost factor of 12 is slow on purpose for security; all other operations, including creating a pickup with model prediction and the WebSocket broadcast, finish within a few tens of milliseconds.

Table 5.5 Performance and Latency Metrics (median of 15 runs)

Operation	Latency
User login (bcrypt verify + JWT)	238.5 ms
Record waste batch	11.3 ms
Create pickup request (with ML prediction)	43.8 ms
Nearby collector search (Haversine)	10.5 ms
Direct ML delay-risk prediction	35.3 ms
Analytics overview (admin)	10.3 ms
Collection history (admin)	9.2 ms
WebSocket GPS broadcast (sender to listener)	4.7 ms

5.4 Comparison with Existing Techniques

We compared the Random Forest with four commonly used classifiers [12], all trained on the same split with the same preprocessing (Table 5.6). The five-fold stratified cross-validation accuracy is given as mean ± standard deviation over the whole dataset.

Table 5.6 Comparison of Classifiers on the Test Set

Model	Accuracy	Precision	Recall	F1	ROC-AUC	5-fold CV Acc.
Logistic Regression	71.4	66.7	68.8	67.7	0.786	70.8 ± 2.0
Gaussian Naive Bayes	73.3	72.3	63.1	67.3	0.782	69.1 ± 1.0
K-Nearest Neighbours (k=7)	69.2	67.2	57.3	61.9	0.751	65.6 ± 2.1
Decision Tree	66.7	62.8	58.0	60.3	0.675	63.7 ± 0.8
Random Forest (proposed)	70.3	66.4	64.3	65.4	0.772	69.1 ± 1.0

The Random Forest (accuracy 70.3 %, ROC-AUC 0.772) clearly did better than the K-Nearest Neighbours (0.751) and the single Decision Tree (0.675) models, which shows the benefit of averaging many trees. Logistic Regression (0.786) and Gaussian Naive Bayes (0.782) scored a little higher, because the synthetic delay label was generated from a logistic (linear) function of the features (Section 4.7). We still kept the Random Forest because real delays are likely to depend on combined effects, such as peak hours together with few free collectors, and because it gives the feature importances shown in the AI Model Centre. The model should be chosen again once real pickup records are available.

The main limitation is that the model was trained on synthetic data. Its accuracy in real use has to be confirmed by retraining it on actual pickup records once the system starts collecting them.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

Conclusion

The Smart Medical Waste Collection System makes biomedical waste collection digital and follows the colour-coding of the Bio-Medical Waste Management Rules. Every pickup request is checked by a Random Forest model that marks it as High, Medium or Low delay risk, so the administrator can handle the most urgent requests first and assign the nearest free collector using Haversine distance. The collector's live position is shown to the hospital through WebSockets, and completed pickups move into a time-stamped history that feeds the analytics dashboards, while role-based access control keeps each user's data separate. The model gave 70.28 % accuracy and an ROC-AUC of 0.772 on unseen data, all automated end-to-end tests passed and the main operations respond within milliseconds.

Future Scope

This work can be extended in the following ways:

- Training on real data – retraining the model regularly using the time stamps recorded by the system, adding the actual distance to the assigned collector and the real waiting times as features, and using the stored GPS traces to estimate travel times [25].
- Route planning with many stops – planning routes that cover several hospitals in one trip using vehicle routing methods that consider vehicle capacity and time windows [22].

REFERENCES

[1] Abdallah, M, Abu Talib, M, Feroz, S, Nasir, Q, Abdalla, H & Mahfood, B 2020, 'Artificial intelligence applications in solid waste management: a systematic research review', *Waste Management*, vol. 109, pp. 231–246.

[2] Anagnostopoulos, T, Zaslavsky, A, Kolomvatsos, K, Medvedev, A, Amirian, P, Morley, J & Hadjieftymiades, S 2017, 'Challenges and opportunities of waste management in IoT-enabled smart cities: a survey', *IEEE Transactions on Sustainable Computing*, vol. 2, no. 3, pp. 275–289.

[3] Breiman, L 2001, 'Random forests', *Machine Learning*, vol. 45, no. 1, pp. 5–32.

[4] Central Pollution Control Board 2018, *Guidelines for Management of Healthcare Waste as per Biomedical Waste Management Rules, 2016*, Central Pollution Control Board, Delhi.

[5] Datta, P, Mohi, GK & Chander, J 2018, 'Biomedical waste management in India: critical appraisal', *Journal of Laboratory Physicians*, vol. 10, no. 1, pp. 6–14.

[6] Fawcett, T 2006, 'An introduction to ROC analysis', *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874.

[7] Fette, I & Melnikov, A 2011, *The WebSocket Protocol*, RFC 6455, Internet Engineering Task Force.

[8] Fielding, RT 2000, 'Architectural styles and the design of network-based software architectures', PhD dissertation, University of California, Irvine.

[9] Govindan, K, Nasr, AK, Mostafazadeh, P & Mina, H 2021, 'Medical waste management during coronavirus disease 2019 (COVID-19) outbreak: a mathematical programming model', *Computers & Industrial Engineering*, vol. 162, article 107668.

[10] Haklay, M & Weber, P 2008, 'OpenStreetMap: user-generated street maps', *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18.

[11] Hannan, MA, Abdulla Al Mamun, M, Hussain, A, Basri, H & Begum, RA 2015, 'A review on technologies and their usage in solid waste monitoring and management systems: issues and challenges', *Waste Management*, vol. 43, pp. 509–523.

[12] Hastie, T, Tibshirani, R & Friedman, J 2009, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd edn, Springer, New York.

[13] He, H & Garcia, EA 2009, 'Learning from imbalanced data', *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284.

[14] Jones, M, Bradley, J & Sakimura, N 2015, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force.

[15] Kannangara, M, Dua, R, Ahmadi, L & Bensebaa, F 2018, 'Modeling and prediction of regional municipal solid waste generation and diversion in Canada using machine learning approaches', *Waste Management*, vol. 74, pp. 3–15.

[16] Kohavi, R 1995, 'A study of cross-validation and bootstrap for accuracy estimation and model selection', *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, vol. 2, pp. 1137–1145.

[17] Ministry of Environment, Forest and Climate Change 2016, *Bio-Medical Waste Management Rules, 2016*, Gazette of India, G.S.R. 343(E), Government of India, New Delhi.

[18] Pedregosa, F, Varoquaux, G, Gramfort, A, Michel, V, Thirion, B, Grisel, O et al. 2011, 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830.

[19] Provos, N & Mazières, D 1999, 'A future-adaptable password scheme', *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*, pp. 81–91.

[20] Sandhu, RS, Coyne, EJ, Feinstein, HL & Youman, CE 1996, 'Role-based access control models', *IEEE Computer*, vol. 29, no. 2, pp. 38–47.

[21] Sinnott, RW 1984, 'Virtues of the haversine', *Sky and Telescope*, vol. 68, no. 2, p. 159.

[22] Toth, P & Vigo, D (eds) 2014, *Vehicle Routing: Problems, Methods, and Applications*, 2nd edn, Society for Industrial and Applied Mathematics, Philadelphia.

[23] World Health Organization 2014, *Safe Management of Wastes from Health-Care Activities*, 2nd edn, eds Y Chartier, J Emmanuel, U Pieper, A Prüss, P Rushbrook, R Stringer, W Townend, S Wilburn & R Zghondi, World Health Organization, Geneva.

[24] Windfeld, ES & Brooks, MSL 2015, 'Medical waste management – a review', *Journal of Environmental Management*, vol. 163, pp. 98–108.

[25] Zheng, Y 2015, 'Trajectory data mining: an overview', *ACM Transactions on Intelligent Systems and Technology*, vol. 6, no. 3, article 29.

APPENDIX 1 – SUSTAINABLE DEVELOPMENT GOALS (SDG)

This project supports four of the United Nations Sustainable Development Goals: SDG 3, SDG 6, SDG 11 and SDG 12.

SDG 3: Good Health and Well-being. This goal aims to ensure healthy lives for all, and Target 3.9 asks for a large reduction in deaths and illnesses caused by hazardous chemicals and pollution. Collecting infectious waste and sharps on time lowers the risk of infection and injury to health-care workers, waste handlers and the public.

SDG 6: Clean Water and Sanitation. Through Target 6.3, this goal aims to improve water quality by reducing pollution and the release of hazardous materials. Biomedical waste that is stored for too long or dumped carelessly can pollute water sources.

SDG 11: Sustainable Cities and Communities. Through Target 11.6, this goal aims to reduce the harmful effect of cities on the environment, with special attention to city and other waste management. Our system gives city authorities a real-time view of waste collection across hospitals, assigns the nearest collector to cut down travel and pollution, and provides data for planning the collection network.

SDG 12: Responsible Consumption and Production. Through Target 12.4, this goal asks for the safe handling of waste throughout its life cycle. Recording every batch by colour category, quantity and source, and keeping a time-stamped history of its collection, supports segregation at the source, accountability and reporting as required by the Bio-Medical Waste Management Rules.

APPENDIX 2 – SOURCE CODE

This appendix lists the main source code of the Smart Medical Waste Collection System: the pipeline used to train the machine learning model and the WebSocket code that sends the live GPS position.

A2.1 Model Training Pipeline

File: ml/train_model.py

```
# Classifier definition: Random Forest Classifier
classifier = RandomForestClassifier(
    n_estimators=150,
    max_depth=12,
    min_samples_split=4,
    min_samples_leaf=2,
    random_state=42,
    class_weight='balanced',
    n_jobs=-1
)


# Full end-to-end Pipeline
model_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', classifier)
])


print("\nTraining Random Forest Classifier on delay risk...")
model_pipeline.fit(X_train, y_train)
```

A2.2 WebSocket Broadcast of Live GPS Positions

File: backend/app/websocket/connection_manager.py

```
async def broadcast_location(self, pickup_id: int, data: Dict[str, Any]):
    self.latest_locations[pickup_id] = data
    if pickup_id in self.active_connections:
        dead_connections = []
        message_text = json.dumps(data)
        for connection in self.active_connections[pickup_id]:
            try:
                await connection.send_text(message_text)
            except Exception:
                dead_connections.append(connection)


        for dead in dead_connections:
            self.disconnect(pickup_id, dead)
```

APPENDIX 3 – APPLICATION SCREENSHOTS

This appendix shows the main screens of the application. They were captured from the running system using the demo data, with a live pickup in progress. The screens are grouped by user role: public pages, hospital portal, administrator portal and collector portal.



Figure A.1 Landing Page

Figure A.1 shows the public landing page.

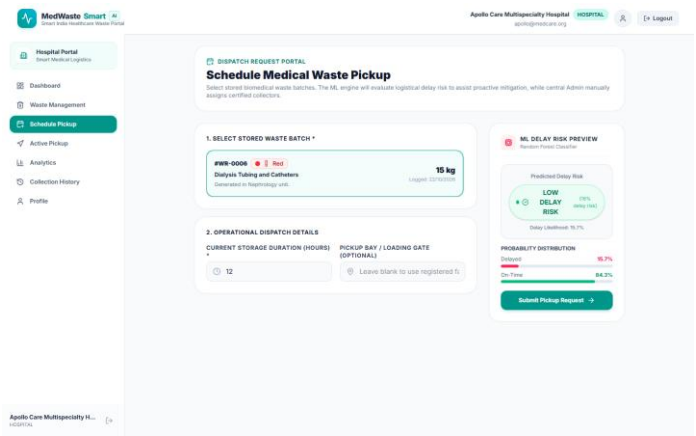


Figure A.2 Hospital – Schedule Pickup with ML Delay-Risk Preview

Figure A.2 shows the pickup scheduling screen.

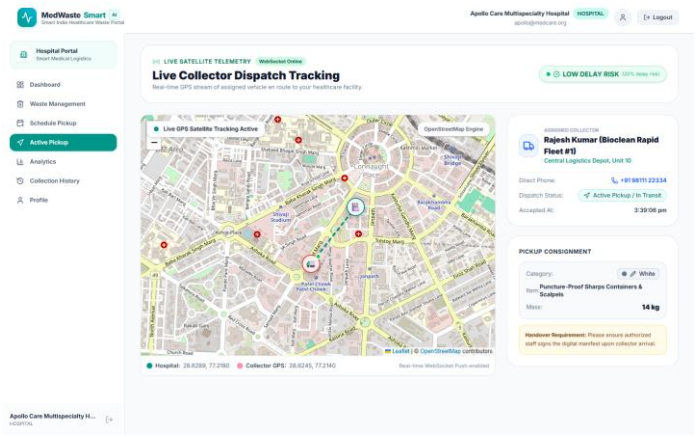


Figure A.3 Hospital – Live Collector Dispatch Tracking

Figure A.3 shows the hospital's live tracking view.

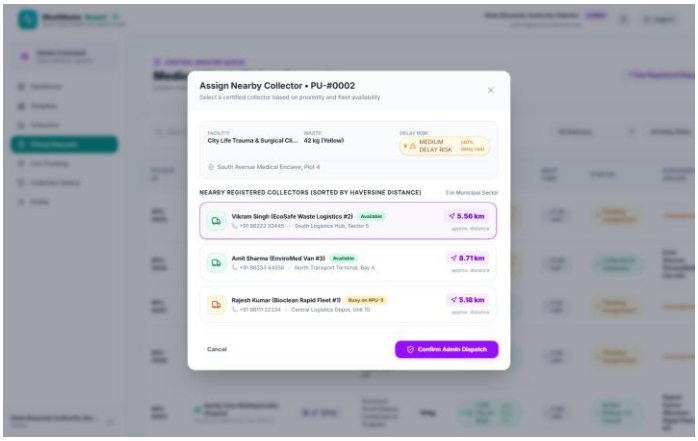


Figure A.4 Admin – Assign Nearby Collector

Figure A.4 shows the assignment dialog for a pending request.

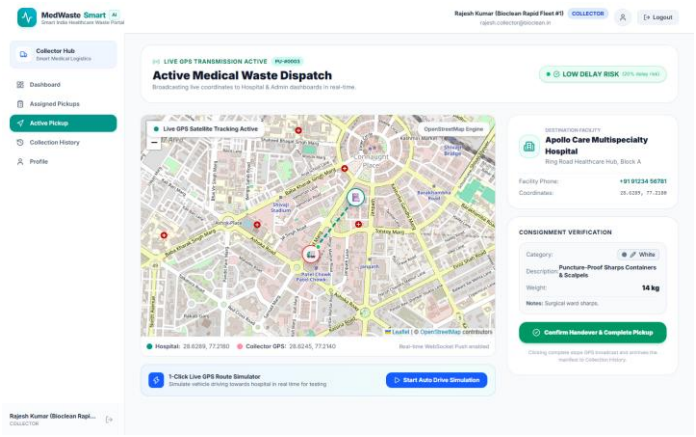


Figure A.5 Collector – Active Medical Waste Dispatch

Figure A.5 shows the collector's active pickup screen.